hashlock.

Security Audit

SwarmBase (Decentralized AI Agent Protocol)

April 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

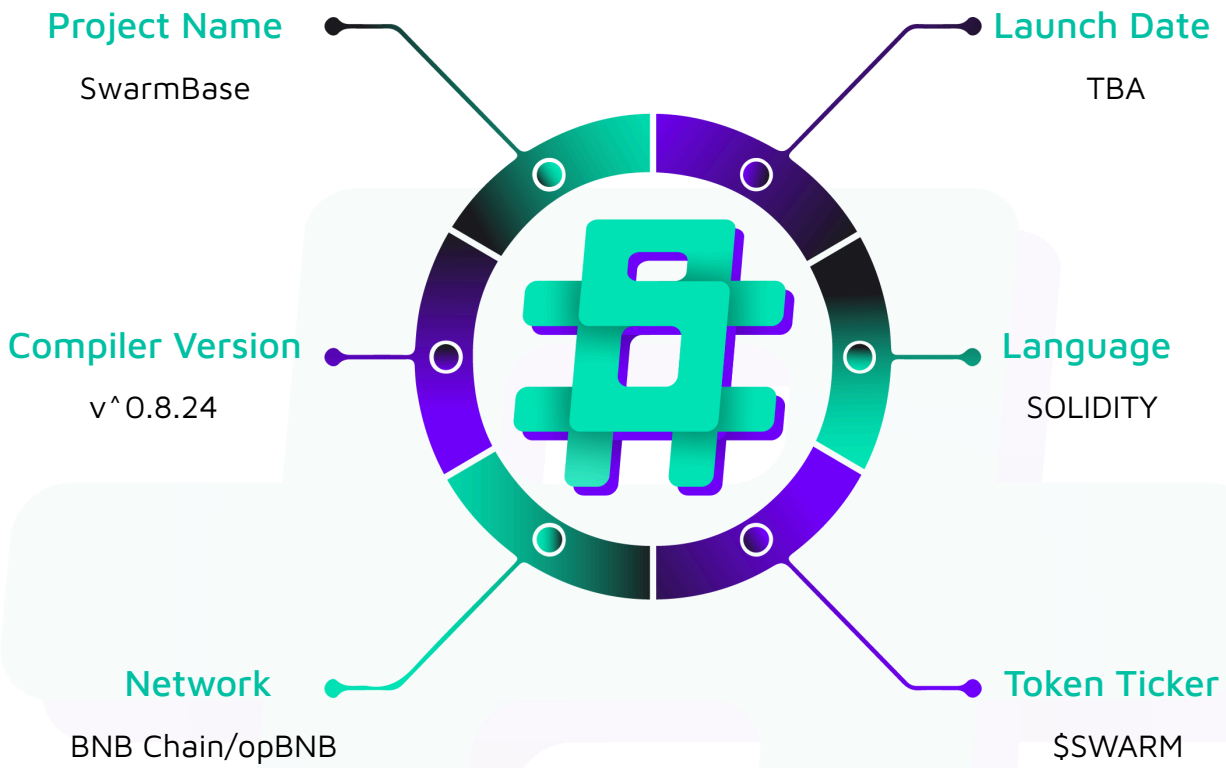
The SwarmBase team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

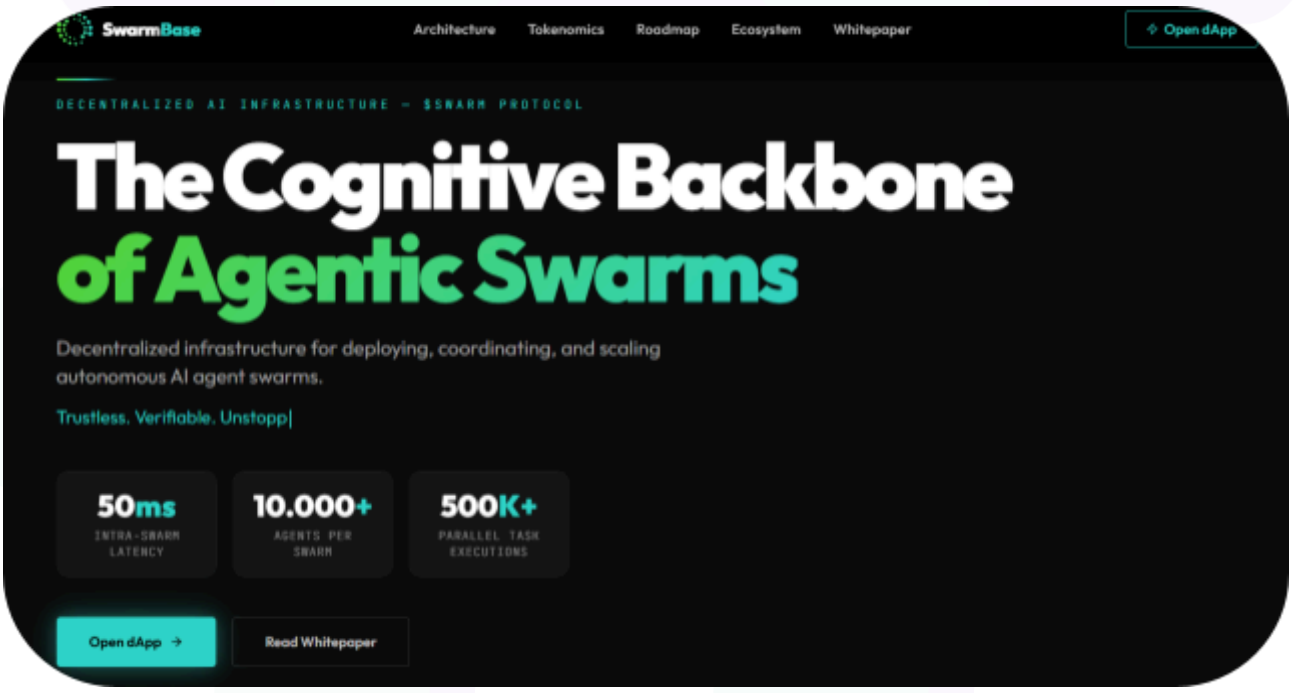
SwarmBase is a decentralized infrastructure protocol enabling autonomous AI agent swarms to coordinate, execute tasks, and operate trustlessly on-chain. It provides the foundational layer for a new class of AI-native applications — where agents can be deployed, orchestrated, and rewarded without centralized control. Participants build on-chain reputation through SwarmScore, earn soulbound NFT badges that reflect their role in the network, and receive $SWARM token rewards tied to real contribution. The protocol is designed to scale alongside the growth of autonomous AI, creating an open, permissionless ecosystem where agents and humans participate together.

Project Name: SwarmBase
Project Type: Decentralized AI Agent Protocol
Compiler Version: ^0.8.24
Website: swarmbase.io
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the SwarmBase project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**SwarmBase Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**April, 2026**
Contract 1	contracts/SwarmCore.sol
Contract 2	contracts/SwarmBadge.sol
Contract 3	contracts/SwarmToken.sol
Contract 4	scripts/deploy.js
Audited GitHub Commit Hash	ecbb079eebb597bdfdfc503cc121da0d80be3074
Fix Review GitHub Commit Hash	8d3d5794599afefcaa641e844463812c1478931e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerability

6 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SwarmCore.sol Is the pre-TGE engagement/points ledger that lets wallets register, check in daily, and accrue an on-chain SwarmScore (a signal, not a token). It also tracks referrals and awards referrers based on referee activity milestones.	**Contract achieves this functionality.**
SwarmBadge.sol Is an ERC-1155 soulbound badge contract that mints 3 tiers of non-transferable badges based on on-chain gates read from SwarmCore.	**Contract achieves this functionality.**
SwarmToken.sol Is the $SWARM ERC-20/BEP-20 compatible token with 1B total supply minted for one time distribution into tokenomics wallets. It includes burning functionality for users and an exclusive one for owner to burn fees and an optional SwarmCore link (settable then lockable).	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the SwarmBase project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the SwarmBase project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] SwarmToken, deploy.js - Total supply minted to deployer EOA

Description

The entire 1,000,000,000 SWARM supply is minted to the deployer's EOA and remains there, unprotected by any multisig, until `distribute()` is called.

Vulnerability Details

The constructor mints `TOTAL_SUPPLY` to the `_owner` argument. The deployment script passes `deployer.address` for this parameter, so all tokens land in a single EOA controlled by one private key. The `distribute()` function must then be called to move tokens to their intended recipients — but this call is a manual post-deployment step with no time constraint enforced on-chain. During the entire interval between deployment and distribution, the full supply is exposed to private-key compromise.

```
// contracts/SwarmToken.sol:85-88
constructor(address _owner) ERC20("SwarmBase", "SWARM") Ownable() {
    _mint(_owner, TOTAL_SUPPLY);
    _transferOwnership(_owner);
}
```

```
// scripts/deploy.js:45
const token = await SwarmToken.deploy(deployer.address);  // EOA as _owner
```

The deploy script itself contains the note: "PRODUCTION DEPLOY: owner must be a Gnosis Safe multisig, not an EOA".

Impact

Compromise of the deployer private key at any point before `distribute()` is called allows an attacker to transfer the entire 1,000,000,000 SWARM supply to an arbitrary address.

Recommendation

Consider implementing the following changes:

1. Accept a dedicated `_treasury` address in the constructor and mint `TOTAL_SUPPLY` to that address rather than to `_owner`. Also, there is not need for the `_transferOwnership(_owner)` call in the constructor as `Ownable` already sets the owner to `msg.sender`.
2. At the end of the deploy script after calling any other functions from the deployer (like `setSwarmCore()`), transfer contract ownership to the same address provided in the constructor. This removes the single private key as custodian of the full supply.

```
constructor(address _treasury) ERC20("SwarmBase", "SWARM") Ownable() {
    require(_treasury != address(0), "Invalid treasury");
    _mint(_treasury, TOTAL_SUPPLY);
}
```

Status

Resolved

Low

[L-01] SwarmBadge#setApprovalForAll - Approval not blocked on soulbound token

Description

The contract enforces soulbound behaviour by overriding `_beforeTokenTransfer`, but does not override `setApprovalForAll`, leaving it callable.

Vulnerability Details

`_beforeTokenTransfer` rejects any transfer where `from != address(0)`, making badges permanently non-transferable. However, `setApprovalForAll` is not overridden, so holders can still grant operator approvals that can never be exercised. The contract emits `ApprovalForAll` events and stores approval state for operators with no ability to act on it.

Impact

No token transfer is possible, but the presence of a functional `setApprovalForAll` produces misleading on-chain state and events that conflict with the contract's soulbound intent.

Recommendation

Override `setApprovalForAll` to revert unconditionally, consistent with the transfer restriction:

```
function setApprovalForAll(address, bool) public pure override {
    revert("SwarmBadge: soulbound - approvals disabled");
}
```

Status

Resolved

[L-02] SwarmBadge#setBaseURI - Duplicate URI storage and missing EIP-1155 URI event

Description

`SwarmBadge` declares its own `_baseURI` storage variable that duplicates ERC1155's internal `_uri`, and `setBaseURI` updates only the shadow copy without calling `_setURI` or emitting the standard `URI` event.

Vulnerability Details

`SwarmBadge` declares `string private _baseURI` alongside the `string private _uri` already managed by ERC1155. The constructor writes `baseURI_` into both slots, through `ERC1155(baseURI_)`, which calls `_setURI`, and then again via `_baseURI = baseURI_`.

EIP-1155 defines a `URI(string value, uint256 indexed id)` event that off-chain consumers rely on to detect metadata changes. `setBaseURI` emits `BaseURIUpdated` instead:

```
function setBaseURI(string memory newURI) external onlyOwner {
    _baseURI = newURI;
    emit BaseURIUpdated(newURI);
}
```

Standard tooling does not monitor `BaseURIUpdated`. Any metadata update will be invisible to external platforms until they re-query the chain.

Impact

Off-chain indexers and NFT marketplaces will not detect metadata URI changes after any update, causing stale metadata to persist in external platforms.

Recommendation

Remove `string private _baseURI` and rely on ERC1155's `_uri` storage exclusively. Update `setBaseURI` to call `_setURI(newURI)` and emit the standard `URI` event for each token ID. Also, update `uri()` to construct the token path from `super.uri(tokenId)` rather than the separate field.

```
function setBaseURI(string memory newURI) external onlyOwner {
    _setURI(newURI);
    emit URI(newURI, PIONEER);
    emit URI(newURI, BUILDER);
    emit URI(newURI, OG);
}


function uri(uint256 tokenId) public view override returns (string memory) {
    require(tokenId >= PIONEER && tokenId <= OG, "SwarmNFT: invalid token ID");
    return string(abi.encodePacked(super.uri(tokenId), tokenId.toString(), ".json"));
}
```

Status

Resolved

[L-03] SwarmBadge#setSwarmCore - Minting allowed while swarmCore is mutable

Description

Badge minting is allowed while `swarmCore` is still mutable. If the owner updates `swarmCore` users who were eligible under the old address may no longer be, or may see different score and registration data after minting started.

Vulnerability Details

All three mint functions read eligibility data exclusively from `swarmCore`: `registered()`, `swarmScore()`, and `registrationTime()`. None of them check `swarmCoreLocked` before proceeding.

Because `setSwarmCore()` only requires `!swarmCoreLocked`, the owner can swap the address at any time before locking. A legitimate update to a new `SwarmCore` deployment could return different values, causing users who qualified under the previous contract to fail eligibility checks or allow users who minted before the update to have badges that they should not have with the new contract data.

Impact

Allowing to mint before locking `swarmCore` can disrupt minting for users who were eligible under the previous address or allow users who minted before the update to have badges that they should not have with the new contract data.

Recommendation

Require `swarmCoreLocked` is set to true before minting is allowed, ensuring the eligibility source is immutable by the time users interact with the badge system.

Status

Resolved

[L-04] SwarmCore#hiveCheckIn - Pause periods silently reset user streaks

Description

The 48-hour streak expiry window advances during paused periods, causing any pause longer than 48 hours to silently reset every active user's streak to zero on the next check-in after unpause.

Vulnerability Details

`hiveCheckIn` is gated by `whenNotPaused`, so users cannot check in while the contract is paused. However, the streak expiry check compares raw `block.timestamp` against `lastHiveCheckIn[msg.sender] + 172800` with no adjustment for time spent paused. A pause lasting more than 48 hours is sufficient to push every active streaker past this threshold.

Impact

Any pause exceeding 48 hours resets all active user streaks to zero, eliminating all levels of multiplier progression that users built through consistent daily check-ins.

Recommendation

Since `pause` is intended exclusively for emergency use, it should be rare and resolved as quickly as possible. Document this behavior clearly so the team is aware that pauses lasting more than 48 hours will reset user streaks, and include an operational guideline to minimize pause duration to avoid affecting streaks.

Status

Resolved

[L-05] SwarmToken#setWallets - Missing event emission on wallet assignment

Description

`setWallets` updates nine distribution wallet addresses without emitting an event, leaving the state change invisible to off-chain systems.

Vulnerability Details

`setWallets` assigns all nine distribution wallets before distribution without emitting an event. Because these wallet addresses determine where the entire token supply flows when `distribute()` is called, any reassignment is invisible to off-chain monitors. The function permits repeated calls before distribution; the only guard is `require(!distributed)`.

Impact

Any silent reassignment of distribution wallet addresses before `distribute()` is called goes undetected, with no on-chain signal available to monitors or stakeholders.

Recommendation

Define a `WalletsSet` event and emit it at the end of `setWallets`.

Status

Resolved

[L-06] SwarmToken#setWallets - setWallets not enforced as one time contrary to documentation

Description

The audit scope documentation states that a `walletsSet` flag prevents `setWallets` from being called more than once, but no such flag exists in the contract.

Vulnerability Details

`docs/AUDIT-SCOPE.md` lists "One-time setWallets — `walletsSet` flag prevents overwrite" as an invariant to verify. The implementation has no `walletsSet` state variable. Both `setWallets` and `updateWallet` share the same single guard `require(!distributed)` meaning:

- `setWallets` can be called any number of times before distribution, overwriting all nine wallet addresses on each call
- `updateWallet` can also be called at any time before distribution (even before any `setWallets` call), and any values set via `updateWallet` can later be overwritten by calling `setWallets` again

Impact

The owner can overwrite all nine distribution wallets an unlimited number of times before calling `distribute()` contrary to the documentation.

Recommendation

If the current implementation is the intended behavior then consider changing the documentation to reflect that. If the current implementation is not the intended behavior then introduce a `walletsSet` boolean and set it to `true` at the end of `setWallets`, then add `require(!walletsSet, "Wallets already set")` to enforce the one time constraint. Subsequent individual address corrections should go through `updateWallet()` and allow calling it only if wallets were already set.

Status

Resolved

QA

[Q-01] All contracts - Unlocked Pragma Version

Description

The contract uses a floating pragma that accepts any Solidity compiler from 0.8.24 upward.

Vulnerability Details

The contracts declares `pragma solidity ^0.8.24`, which does not bind compilation to a specific version. This allows contracts to be compiled with different compiler versions than those tested. Future compiler versions may introduce semantic changes, new defaults, or undiscovered bugs that alter contract behavior.

Impact

The contract may be compiled with an unintended Solidity version, introducing compiler level behavior changes that were not present during the audit.

Recommendation

Lock the pragma to the specific compiler version used during development: `pragma solidity 0.8.24`

Status

Resolved

[Q-02] SwarmToken#receive - Redundant receive() function

Description

The contract defines a `receive()` function whose sole purpose is to revert, but this is already the default behavior for any contract that lacks a `receive()` or payable `fallback()`.

Vulnerability Details

`SwarmToken` inherits from OpenZeppelin's `ERC20`, which defines no `receive()` and no payable `fallback()`. Without either, the EVM reverts any plain ETH transfer automatically — no explicit revert is needed.

Impact

Deployment cost and code size is marginally increased with no change in behavior.

Recommendation

Remove the `receive()` function.

Status

Resolved

Centralisation

The SwarmBase project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the SwarmBase project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au